Modules 5&6 Part 2 - Summary Video – Transcript
Note: This transcript is auto-generated from the Teams application; it may include errors.

Hello everyone and welcome. I am Maria Konte, the instructor of the Computer Networks course, and this is Part 2 on router design and algorithms. In Part 1, we started taking a look at the functionalities that the router has, and we started zooming into each one of these in more detail.

Now we're continuing with longest prefix matching. Routers do not store one entry per destination IP address. Instead, the forwarding table stores destination prefixes like a /24 or a /16. So when a packet arrives, the router checks which prefixes match the destination address.

Often we may have more than one prefix match. For example, a broad prefix might cover a whole organization, while a more specific prefix covers a specific subnet. In that case, the router chooses the longest prefix match, meaning the most specific prefix that still matches the destination. This is also what lets routing scale, because we can keep a general route for a large block, but override it with more specific ones when needed.

So the next question is how do we implement the lookup efficiently? One classic data structure is a trie. A trie is essentially a prefix tree. Keys that share the same starting characters share the same path near the root. The word examples here, such as cab, cat, and code, show the idea, so we can walk down following the letters and common prefixes reuse the same nodes. For IP routing, we do the exact same idea but with bits instead of letters. Each step in the trie consumes the next bit of the destination address and takes us left or right. As we walk, we pass nodes that represent valid prefixes, and those are routing table entries. To implement longest prefix matching, the router keeps track of the most recent matching prefix it has seen so far on the path.

If the walk ends because there's no deeper match, the router returns the last recorded prefix as the best match.

So now let's translate the trie idea more specifically for IP prefix by looking at an example. On the left we have prefixes such as one star, zero star, 101 star, and so on. The star just means all addresses that start with those bits.

A unibit trie stores these prefixes in a binary tree from the root. Taking the zero edge means the next address bit is 0. Taking the one edge means it's one. So to look up a destination address, we read its bits from left to right and follow the corresponding edges down the trie.

As we move, we might hit nodes that correspond to valid prefixes. These are potential forwarding matches. So just like before, we keep the best so far, meaning the most specific prefix that we have encountered so far. So when we're not able to go any further, we return that saved prefix as the longest prefix match. The tradeoff is performance and memory in this case. In the worst case, the walk can take up to the address width, which is 32 steps for an IPv4 address.

Now let's take a look at an optimization, which is multibit tries. So instead of branching on one bit, we choose a stride. Here, for example, we have two bits as our stride, so each level branches on the next two bits of the destination address.

So that's why each node has four possible next choices, such as 00, 01, 10, and 11. During lookup, we take the destination IP, we read the next two bits, we jump to the matching child, and then read the next two bits, and so on.

So we can cut down the number of memory accesses because we're walking fewer levels to reach the same specificity. The cost, though, is that each node is wider, so some of those four-way entries might be empty, and as we increase the stride, the table can become more sparse and use more memory.

Multibit tries are a classic speed-memory tradeoff.

So we just saw how to speed up lookup by using a multibit trie, taking two bits per step. But the issue that we are running into here is what do we do with prefixes that don't line up nicely with the stride that we have selected. So this is where prefix expansion comes in.

Suppose that our stride is 2 bits, but we have a prefix such as 11 star. That prefix is only two bits long and then anything represented by the star. In a stride-based table, we want entries that match the table's granularity, so we expand 11 star into all the more specific prefixes that start with 11 at the next stride boundary, so 1100 star, 1101 star, 1110 star, and 1111 star. So same idea on the left. A prefix like 100 star can be expanded to 100 star.

The upside is speed, so once prefixes are expanded, lookup can be a simple index by the next stride bits. The downside, though, is memory because expansion can multiply the number of prefixes that we need to store.

Now let's try prefix expansion for a three-bit multibit trie. Our goal is again to make every prefix line up with a three-bit stride. So we expand prefixes until their length is a multiple of three. If a prefix is shorter than the next multiple of three, then we append bits and generate all possible combinations to fill out the missing bits, and if any expanded version overlaps with an existing more specific prefix, we drop the expanded copy because the more specific prefix still wins.

So here first P1 and P2, which are 111 star, already have a length of three, so they stay the same. Next, P3 has a length of five. To reach the next multiple of three, or length 6, we add one extra bit and include both possibilities.

So P3 becomes 110010 star and 110011 star. Now P4 has a length of one, so to make it a length of three we append 2 bits, so it expands to 100 star, 101 star, 110 star, and 111 star, but those prefixes already exist in our database. So here we have a collision. So in this case, we drop the expanded copies because we don't want expansion to override a more specific entry. For P5, the expansion logic is the same, and this gives us 000 star, 001 star, 010 star, and 011 star. For P6, we expand to length six, but the expansion of 100000 star collides because it already exists as P7, so we drop that one, and finally P8 and P9 do not need an expansion.

Now we're changing context, and we are shifting from lookup to scheduling. On a busy router, scheduling decisions happen in real time. Packets arrive back-to-back, and the router has to decide which packets to serve next at extremely small time scales.

The simplest approach is first in, first out with tail drop. In other words, packets are forwarded to an output port, placed in a first-in, first-out queue, and if the buffer is full, new arrivals at the tail get dropped. This is an easy and fast approach, but it treats all packets the same and can drop or delay important traffic during bursts. This is why we need quality-of-service-aware scheduling, which means that under congestion we want better throughput and stability, and we want to prevent one backup flow from freezing interactive traffic.

We also want fair sharing, so competing flows each get a reasonable bandwidth or delay bound.

The problem that we have with first-in, first-out queues is the head-of-line blocking problem. Imagine that each input port has just one first-in, first-out queue. If the packet at the front wants to exit through a busy output, it sits there and it can block packets behind it even if those packets are destined to outputs that are currently free, so the switch fabric looks idle even though there is traffic that could have moved in the meantime. One fix is virtual output queues. Instead of one first-in, first-out per input, each input maintains multiple virtual queues, one for each output. Now if output 2 is congested, only the queue for output 2 builds up. Packets destined for outputs 1, 3, and 4 can still move forward. This sets up efficient crossbar scheduling because once packets are separated by destination, we can try to pick a set of input-output transfers that don't conflict and run them in parallel during each time slot.

Now that we have seen how virtual output queues can help, we need a fast way to schedule the crossbar. Here we see one such technique that is called parallel iterative matching, often described as request, grant, accept.

So in step one, which is the request, each input looks at its virtual output queues and requests the output it wants to send to. At Step 2, which is the grant, each output may see multiple requests, so it chooses one to grant. And at the final and third step, which is the accept step, an input may receive more than one grant, but it can only send one packet per time slot, so it accepts at most one grant. After these three phases, we get a set of connections where no input or output is used more than once, which is a valid matching, so several packets can cross the fabric in parallel.

Routers repeat this process quickly, sometimes multiple iterations per slot, in order to improve utilization.

Now let's address one additional problem that shows up. So with virtual output queues, each input has multiple queues, and in a given time slot we can have many inputs requesting the same output. So the fabric, though, still needs to make a decision about who gets to cross right now.

So this is the role of a ticketing scheduling algorithm. That conflict is resolved using a ticketing mechanism. The high-level goal is to compute conflict-free matching for the crossbar every time slot, meaning each input connects to at most one output and each output connects to at most one input. So we want that matching to be fast to compute and keep utilization high and avoid starving any flows or any input outputs over time. So round one works like this. First, in the request step, each input looks at the head packet in its virtual output queues and requests the corresponding output. Next, in the ticket grant, each output sees a set of requests and uses the ticket rule. Think of it as a "who's next in line" type of

ticket to pick one request to grant. And finally, in the connect step, the crossbar actually makes those granted connections, and those packets get to cross the switch.

After the first ticket grant round, we usually don't match everyone because some inputs may have lost the contention or some outputs may already be taken. So we do round two. The unmatched inputs submit the remaining requests again, and outputs again grant one request based on the ticket rule. Then we connect the new set of nonconflicting pairs. So the key idea is that we are building up a better matching iteratively. We don't solve a huge optimization problem, we just do a few quick rounds that are easy to implement in hardware, and then in round three it is the same pattern again. So request, ticket, grant, and then connect.

Each round tries to add more nonconflicting input-output connections, so the crossbar carries more packets in parallel instead of leaving ports idle. Now one more question to ask is why we do not run rounds forever?

The answer is because each additional iteration costs time and hardware complexity. Real routers choose a small number of iterations that give most of the throughput benefit without making the scheduler too slow. So in summary, virtual output queues eliminate head-of-line blocking, and then ticket-based iterative scheduling is one practical way to keep the crossbar busy at high speeds.

So up to now we have seen lookup from the perspective of matching the destination prefix and picking an output port. The problem, though, is that real networks want more than answers such as where does this packet go next. They also care about what kind of traffic this packet is related to.

So for quality-of-service purposes, for example, or enforcing security policies or performing traffic measurement tasks and so on. So the goal of packet classification is to map a packet into a policy class and then apply the right action. For example, choose a queue or priority, mark it, apply rate limiting, or even drop it. In this picture, for example, we have router R that sits at several connection points and sees traffic headed to different destinations, and possibly those could be headed towards different links.

For packet classification, it uses multiple header fields, not only the destination IP address. For example, it can use source/destination IP, it can use source/destination ports, TCP flags, or other fields.

Now on this figure, we see how set pruning supports two-dimensional classification, so that is based on matching both the destination and the source prefixes. At the top we have a destination trie, so we take the packet's destination IP, we read its bits, and we walk down the tree, zero branches on the left and one branches on the right, until we land at the destination region that matches best. Now here is the set pruning idea. Instead of carrying all possible rules with us, each destination node points to a smaller structure below, which is a source trie.

So these orange triangles that we see in the picture are source tries, and each one contains only one subset of source prefix rules that are relevant for that destination region. So we can see the subsets and how they are different because one destination node might keep a big set like, for example, S1, S5, and S7, while another one might only have S7, and another one might have only S6 and S7. Therefore, we cut down or prune, as we say, the candidate set early by using destination bits.

After the destination walk selects the correct triangle, then we do a source prefix lookup inside the source trie using the packet source IP address.

Now, with set pruning, as we just discussed, we attach a source trie to many destination nodes, which makes lookup faster, but it costs us in terms of memory. The backtracking approach is the opposite tradeoff or offers the opposite tradeoff because we try to store fewer copies of those structures. So here is how the lookup works in the case of backtracking. First we walk down the destination trie using the destination IP bits, just like before, to reach the most specific destination region. Then we try the source trie stored at the destination node using the source IP bits.

If we find a match, then we are done. But if we don't, we don't give up because less specific destination prefixes might still have a rule that matches the source that we have. Therefore we backtrack, or in other words, we move up to the parent destination node and we try its source trie. If that doesn't lead to a match, then we will need to potentially move up again until either we find a match or, worst case, we hit, you know, we go back to the root. So backtracking saves memory by not storing large candidate sets everywhere. But it can require multiple source lookups per packet.

In summary, on the left we have the set pruning tries, so we first walk the destination trie using destination bits. At the node that we land, we jump into the source trie that contains only the relevant candidate rules for that destination region.

In the common case, it's basically one clean pipeline because we do the destination walk, then we do the source walk. So it's fast, but that means that we pay more memory because we store multiple copies of source structures. On the right-hand side, we have the backtracking approach.

So that saves us memory, and in order to save memory we store fewer source tries. But if we don't find a match at the most specific destination node, we may need to move up to a less specific destination prefix and try again, possibly several times.

This extra work means that lookup is slower.

Now in this slide we show a third approach, which is the grid of tries, which reduces time by using recomputation. The key idea is that it uses switch pointers, which means that when we hit a failure point inside the source trie, the data structure already knows the next possible source trie that could contain a better matching rule. Instead of backtracking to the ancestors and retraversing again, we jump directly to the next possible candidate.

So as an example, let's assume that we have the destination address, which is 001, and the source address, which is 001. So we start in the destination trie and we get the best match, say at D, which is 00. Next we try the associated source trie, and we hit a failure. Instead of backtracking, we follow the switch pointer, which is labeled 0 in this figure, to the next candidate node, which is labeled as X. It fails again, so we follow the next switch pointer to Y, to node Y. At that point, we find the match and we stop the lookup.

Throughout this, we still track our current best source match, and the switch pointers help us skip source tries that can't beat what we already have, especially those with shorter source prefixes.

Now up to this point, we have focused on packet classification, which is how routers match packets to rules efficiently, even when rules depend on both the destination and source, like on multiple filters. But once a router classifies traffic, the next question is what do we do with that classification?

One common action is rate control. In other words, limiting how much traffic a class is allowed to inject into the network, making sure a flow doesn't exceed what the network can

handle, because bursty traffic can quickly fill buffers, and that creates delay and then perhaps packet drops.

So we want mechanisms that enforce a traffic profile. So we have two main approaches, traffic shaping and traffic policing. Traffic shaping smooths the stream. If packets arrive faster than allowed, the routers hold the excess in a buffer and send them later. So shaping trades extra delay for fewer packet drops and a steady rate. On the other hand, we have traffic policing, which is stricter. So if the flow exceeds the configured rate, the excess packets are dropped or sometimes marked for lower priority, so that keeps the network protected, but it can increase packet loss.

So now that we have defined what is shaping versus policing, the next question that we ask is how does a router decide whether a packet is within the limit or if it is excess? A common mechanism is the token bucket.

Token bucket is a standard way to enforce both an average rate and a burst limit. So to be more specific, each flow has a bucket that fills with tokens at rate R and can hold at most B tokens. If the bucket is full, extra tokens are discarded, so credit never grows beyond B. When a packet arrives, it needs tokens that are proportional to its size. If there are enough tokens, it's conforming and goes out immediately. If there aren't enough tokens, then the behavior depends on the goal that we have.

More specifically, for token bucket shaping, the packet waits until enough tokens accumulate, so we smooth traffic by delaying it. On the other hand, for token bucket policing, the packet is dropped or marked right away, so the rate limiting is enforced strictly.

Now, as we just discussed, token bucket controls rate by spending tokens, which allows bursts when tokens have accumulated. On the other hand, the leaky bucket approach is a slightly different approach in the sense that it produces a steady output rate.

So think of a bucket with capacity B that holds arriving packets, and the bucket leaks at a specific fixed rate R. So no matter how bursty the input is, packets are released at that constant rate, which smooths the traffic entering the network.

If arrivals come in faster than the leak rate for too long, the bucket fills, and at that point what we do with that depends on the mechanism or the goal that we have. Again, thinking about policing or shaping, for policing, new arrivals are dropped when the bucket is full.

Whereas for shaping, we are effectively buffering and delaying until space becomes available.

Now on this slide we compare two common rate control tools, which are, as we discussed, the token bucket and the leaky bucket. So with the token bucket, the key idea is that tokens accumulate over time. If a flow has been quiet, it can build up tokens and then send a burst.

When it needs to, up though to a specific burst limit B, as shown in the previous figure that we talked about. This mechanism will still keep the long-run average rate bounded by R. So token bucket is burst tolerant but controlled. On the other hand, a leaky bucket is smoother, so packets may arrive in bursts, but they have a near-constant rate R, so the network sees a steadier stream rather than spikes.

In summary, in this video, we saw that routers split their operations into a fast data plane that forwards at line rate and a slower control plane that computes and installs forwarding state. Inside the data plane, forwarding resembles a pipeline where we have lookup, switching, queuing, scheduling, and then output, and contention in that pipeline creates delay and loss. We also looked at how fast lookup depends on longest prefix matching and efficient structures such as unibit and multibit tries with prefix expansion as a speed-memory tradeoff.

For crossbar switches, we saw why scheduling matters because head-of-line blocking motivates virtual output queues and schedulers using, for example, as we saw, request, grant, accept, and ticket-based mechanisms. Finally, modern routers enforce policy. Packet classification supports quality of service and security, and rate control uses policing versus shaping with token bucket and leaky bucket mechanisms.

Thank you for watching!